📘 Lists in Python

List Operations | All Built-in List Methods/Functions

Introduction to Lists

Definition: A list is a sequence of elements enclosed in square brackets (`[ ]`), where elements can be of different data types.

```
list1 = [10, 20, 30, 40]
list2 = ["apple", "banana", "cherry"]
list3 = [1, "hello", 3.14, True]
list4 = []  # empty list
```

Important Points:

- Lists are mutable (can be changed after creation)
- Elements in a list can be accessed using index positions
- Indexing starts from 0 (forward) and -1 (backward)
- Lists can contain elements of different data types
- Lists maintain the order of elements (ordered collection)

List Operations

1. Concatenation (+)

Purpose: Joins two or more lists together

Syntax: `list1 + list2`

Parameters: Two or more lists

Return: A new list combining all lists

- Both operands must be lists
- Creates a new list (doesn't modify original)

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
```

```
result = list1 + list2  # [1, 2, 3, 4, 5, 6]
```

2. Repetition / Replication (*)

Purpose: Repeats a list a specified number of times

Syntax: `list * n` or `n * list`

Parameters: list (compulsory), integer n (compulsory) — number of repetitions

Return: A new list repeated n times

- n must be an integer
- If n = 0, returns empty list

```
list1 = [1, 2]
result = list1 * 3  # [1, 2, 1, 2, 1, 2]
```

3. Indexing

Purpose: Accesses an individual element of a list using its position

Syntax: `list[index]`

Parameters: index (compulsory) — position of the element

Return: A single element at the given index

- Forward indexing starts from 0 (left to right)
- Backward indexing starts from -1 (right to left)
- Index must be an integer, otherwise `TypeError` occurs
- Raises `IndexError` if the index is out of range

```
list1 = [10, 20, 30, 40, 50]
print(list1[0])    # 10 (first element)
print(list1[4])    # 50 (last element)
print(list1[-1])   # 50 (last element, backward index)
print(list1[-5])   # 10 (first element, backward index)
# print(list1[10])  # IndexError: list index out of range
```

4. Slicing

Purpose: Extracts a portion of a list

Syntax: `list[start:end:step]`

Parameters:

- start (optional) — starting index (default: 0)
- end (optional) — ending index (excluded, default: length)
- step (optional) — step value (default: 1)

Return: A new sublist

- All parameters are optional
- Negative indices can be used
- start < end for forward slicing

```
list1 = [10, 20, 30, 40, 50, 60]
print(list1[0:3])      # [10, 20, 30]
print(list1[3:])       # [40, 50, 60]
print(list1[::-1])     # [60, 50, 40, 30, 20, 10] (reverse)
print(list1[::2])      # [10, 30, 50]
```

5. Membership Operators (in / not in)

Purpose: Checks if an element exists in a list

Syntax: `element in list` or `element not in list`

Parameters: element (compulsory) — item to search for

Return: Boolean (True/False)

- Case-sensitive for string elements
- `in` returns True if element is present
- `not in` returns True if element is not present

```
list1 = [10, 20, 30, 40]
print(20 in list1)       # True
print(50 in list1)       # False
print(50 not in list1)   # True
```

Traversing a List Using Loops

Using for loop:

```
list1 = [10, 20, 30, 40]
for item in list1:
    print(item)
# Output: 10 20 30 40 (each on new line)
```

Using for loop with index:

```
list1 = [10, 20, 30, 40]
for i in range(len(list1)):
    print(list1[i])
```

Using while loop:

```
list1 = [10, 20, 30, 40]
i = 0
while i < len(list1):
    print(list1[i])
    i += 1
```

Built-in Functions / Methods

1. len()

Purpose: Returns the number of elements in a list

Syntax: `len(list)`

Parameters: list (compulsory)

Return: Integer (number of elements)

- Counts all elements in the list

```
list1 = [10, 20, 30, 40, 50]
print(len(list1))  # 5
```

2. list()

Purpose: Creates a list from an iterable (string, tuple, range, etc.)

Syntax: `list(iterable)`

Parameters: iterable (optional) — any iterable object

Return: A new list

- If no argument given, returns an empty list
- Converts string characters into list elements

```
print(list())              # []
print(list("Hello"))       # ['H', 'e', 'l', 'l', 'o']
print(list(range(5)))      # [0, 1, 2, 3, 4]
```

3. append()

Purpose: Adds a single element at the end of the list

Syntax: `list.append(element)`

Parameters: element (compulsory) — item to be added

Return: None (modifies the original list)

- Adds only one element at a time
- The element can be of any data type
- Modifies the original list in place

```
list1 = [10, 20, 30]
list1.append(40)
print(list1)  # [10, 20, 30, 40]
```

4. extend()

Purpose: Adds all elements of an iterable to the end of the list

Syntax: `list.extend(iterable)`

Parameters: iterable (compulsory) — list, tuple, string, etc.

Return: None (modifies the original list)

- Adds multiple elements at once
- If a string is passed, each character is added separately
- Modifies the original list in place

```
list1 = [10, 20, 30]
list1.extend([40, 50])
print(list1)  # [10, 20, 30, 40, 50]

list2 = [1, 2]
list2.extend("AB")
print(list2)  # [1, 2, 'A', 'B']
```

5. insert()

Purpose: Inserts an element at a specified position in the list

Syntax: `list.insert(index, element)`

Parameters:

- index (compulsory) — position where element is to be inserted
- element (compulsory) — item to be inserted

Return: None (modifies the original list)

- If index is out of range, element is added at the end
- Modifies the original list in place

```
list1 = [10, 20, 40, 50]
list1.insert(2, 30)
print(list1)  # [10, 20, 30, 40, 50]

list1.insert(0, 5)
print(list1)  # [5, 10, 20, 30, 40, 50]
```

6. count()

Purpose: Counts occurrences of an element in a list

Syntax: `list.count(element)`

Parameters: element (compulsory) — item to count

Return: Integer (number of occurrences)

- Case-sensitive for string elements
- Returns 0 if element not found

```
list1 = [10, 20, 10, 30, 10, 40]
print(list1.count(10))  # 3
print(list1.count(50))  # 0
```

7. index()

Purpose: Finds the first occurrence of an element

Syntax: `list.index(element, start, end)`

Parameters:

- element (compulsory) — item to search for
- start (optional) — starting index (default: 0)
- end (optional) — ending index (default: length)

Return: Integer (index of first occurrence)

- Raises ValueError if element not found
- Case-sensitive for string elements

```
list1 = [10, 20, 30, 20, 40]
print(list1.index(20))      # 1
print(list1.index(20, 2))   # 3
# print(list1.index(50))    # ValueError
```

8. remove()

Purpose: Removes the first occurrence of an element from the list

Syntax: `list.remove(element)`

Parameters: element (compulsory) — item to be removed

Return: None (modifies the original list)

- Removes only the first occurrence
- Raises ValueError if element not found
- Modifies the original list in place

```
list1 = [10, 20, 30, 20, 40]
list1.remove(20)
print(list1)  # [10, 30, 20, 40]
```

9. pop()

Purpose: Removes and returns the element at a specified index

Syntax: `list.pop(index)`

Parameters: index (optional) — position of element (default: last element)

Return: The removed element

- If no index given, removes the last element
- Raises IndexError if list is empty or index out of range
- Modifies the original list in place

```
list1 = [10, 20, 30, 40, 50]
print(list1.pop())     # 50 (removes last element)
print(list1)           # [10, 20, 30, 40]
print(list1.pop(1))    # 20 (removes element at index 1)
print(list1)           # [10, 30, 40]
```

10. reverse()

Purpose: Reverses the order of elements in the list

Syntax: `list.reverse()`

Parameters: No parameters

Return: None (modifies the original list)

- Reverses the list in place
- Does not return a new list

```
list1 = [10, 20, 30, 40]
list1.reverse()
print(list1)  # [40, 30, 20, 10]
```

11. sort()

Purpose: Sorts the elements of the list in ascending order

Syntax: `list.sort(reverse=True/False)`

Parameters: reverse (optional) — if True, sorts in descending order (default: False)

Return: None (modifies the original list)

- Sorts the list in place
- All elements must be of comparable types
- Default order is ascending

```
list1 = [40, 10, 30, 20]
list1.sort()
print(list1)  # [10, 20, 30, 40]

list1.sort(reverse=True)
print(list1)  # [40, 30, 20, 10]
```

12. sorted()

Purpose: Returns a new sorted list without modifying the original list

Syntax: `sorted(list, reverse=True/False)`

Parameters:

- list (compulsory) — the list to be sorted
- reverse (optional) — if True, sorts in descending order (default: False)

Return: A new sorted list

- Does not modify the original list
- Can be used with any iterable

```
list1 = [40, 10, 30, 20]
new_list = sorted(list1)
print(new_list)  # [10, 20, 30, 40]
print(list1)     # [40, 10, 30, 20] (unchanged)
```

13. min()

Purpose: Returns the smallest element in the list

Syntax: `min(list)`

Parameters: list (compulsory)

Return: The smallest element

- Works on numbers and strings (lexicographic)
- Raises ValueError if list is empty

```
list1 = [40, 10, 30, 20]
print(min(list1))  # 10
```

14. max()

Purpose: Returns the largest element in the list

Syntax: `max(list)`

Parameters: list (compulsory)

Return: The largest element

- Works on numbers and strings (lexicographic)
- Raises ValueError if list is empty

```
list1 = [40, 10, 30, 20]
print(max(list1))  # 40
```

15. sum()

Purpose: Returns the sum of all numeric elements in the list

Syntax: `sum(list)`

Parameters: list (compulsory) — must contain only numbers

Return: The sum of all elements

- Raises TypeError if list contains non-numeric elements
- Returns 0 for an empty list

```
list1 = [10, 20, 30, 40]
print(sum(list1))  # 100
```

Nested Lists

Definition: A nested list is a list that contains other lists as its elements.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Accessing elements in nested lists:

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix[0])      # [1, 2, 3]
print(matrix[1][2])   # 6
print(matrix[2][0])   # 7
```

Traversing a nested list:

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for row in matrix:
    for element in row:
        print(element, end=" ")
    print()
# Output:
# 1 2 3
# 4 5 6
# 7 8 9
```

- Nested lists are used to represent matrices, tables, and grids
- Each inner list can be accessed using its index
- Double indexing is used to access individual elements

Suggested Programs

1. Finding the Maximum, Minimum, and Mean of Numeric Values

Purpose: Find max, min, and mean of numbers stored in a list

```
numbers = [25, 10, 45, 30, 15]

# Using built-in functions
maximum = max(numbers)
minimum = min(numbers)
mean = sum(numbers) / len(numbers)

print("Maximum:", maximum)   # 45
print("Minimum:", minimum)   # 10
print("Mean:", mean)         # 25.0
```

2. Finding Maximum, Minimum, and Mean Without Built-in Functions

```
numbers = [25, 10, 45, 30, 15]

maximum = numbers[0]
minimum = numbers[0]
total = 0

for num in numbers:
    if num > maximum:
        maximum = num
    if num < minimum:
        minimum = num
    total += num

mean = total / len(numbers)

print("Maximum:", maximum)   # 45
print("Minimum:", minimum)   # 10
print("Mean:", mean)         # 25.0
```

3. Linear Search on a List of Numbers

Purpose: Search for an element in a list by checking each element one by one

```
numbers = [10, 25, 30, 45, 50]
search = 30
found = False
position = -1

for i in range(len(numbers)):
    if numbers[i] == search:
        found = True
        position = i
        break

if found:
    print(search, "found at index", position)
else:
```

```
    print(search, "not found")
# Output: 30 found at index 2
```

4. Counting the Frequency of Elements in a List

Purpose: Count how many times each element appears in a list

```
numbers = [10, 20, 10, 30, 20, 10, 40]

# Method 1: Using count()
visited = []
for num in numbers:
    if num not in visited:
        print(num, "occurs", numbers.count(num), "times")
        visited.append(num)

# Output:
# 10 occurs 3 times
# 20 occurs 2 times
# 30 occurs 1 times
# 40 occurs 1 times
```

5. Counting Frequency Using Dictionary

```
numbers = [10, 20, 10, 30, 20, 10, 40]

frequency = {}
for num in numbers:
    if num in frequency:
        frequency[num] += 1
    else:
        frequency[num] = 1

for key, value in frequency.items():
    print(key, "occurs", value, "times")
```

6. Counting Frequency Using Nested Loops

```
numbers = [10, 20, 10, 30, 20, 10, 40]
n = len(numbers)
```

```
counted = [False] * n

for i in range(n):
    if counted[i]:
        continue
    count = 1
    for j in range(i + 1, n):
        if numbers[i] == numbers[j]:
            count += 1
            counted[j] = True
    print(numbers[i], "occurs", count, "times")
```

Quick Reference Table

Method / Function	Purpose	Returns
`len()`	Number of elements	Integer
`list()`	Create list from iterable	List
`append()`	Add element at end	None
`extend()`	Add multiple elements	None
`insert()`	Insert at position	None
`count()`	Count occurrences	Integer
`index()`	Find position	Integer
`remove()`	Remove first occurrence	None
`pop()`	Remove and return element	Element
`reverse()`	Reverse the list	None
`sort()`	Sort in place	None
`sorted()`	Return sorted list	List
`min()`	Smallest element	Element
`max()`	Largest element	Element
`sum()`	Sum of elements	Number

Important Board Exam Tips Must Know

1. **List Mutability:** Lists can be changed in place. Methods like `append()`, `insert()`, `remove()`, `sort()` modify the original list.
2. **Indexing:** Remember that indexing starts from 0 in Python.
3. **Negative Indexing:** -1 is the last element, -2 is second last, etc.
4. **Slicing Rules:**
   - `list[start:end]` — includes start, excludes end
   - `list[:end]` — from beginning to end-1
   - `list[start:]` — from start to end
   - `list[::-1]` — reverses the list
5. **append() vs extend():**
   - `append()`: Adds a single element (can be a list as one element)
   - `extend()`: Adds each element of an iterable separately
6. **remove() vs pop() vs del:**
   - `remove(x)`: Removes first occurrence of value x
   - `pop(i)`: Removes and returns element at index i
   - `del list[i]`: Deletes element at index i without returning
7. **sort() vs sorted():**
   - `sort()`: Modifies the original list, returns None
   - `sorted()`: Returns a new sorted list, original unchanged
8. **List Concatenation and Repetition:**
   - `list1 + list2`: Creates a new list with all elements
   - `list * n`: Repeats the list n times
9. **Nested Lists:** Use double indexing like `matrix[i][j]` to access elements.
10. **Common Errors to Avoid:**
    - Forgetting that `append()` takes only one argument
    - Using `remove()` with an element not in the list (raises ValueError)
    - Confusing `sort()` and `sorted()`
    - Forgetting that lists are mutable (changes affect the original)
    - Using `index()` on an element not in the list (raises ValueError)